KuberSelfHeal

Diagnosing and self-healing cross-node pod connectivity failures on a Raspberry Pi Kubernetes cluster

Hem Tilva
24110374

Vansh Barfiwala
24110380

Pratham Choksi
24110275

Shivangi Thaker
24110371

Jils Shah
24110148

One symptom, many possible causes

The backend on node A cannot reach its database on node B.

Every request crosses several layers. A mistake in any one of them looks the same to the user.

Kubernetes restarts crashed pods. It does **not** fix a configuration that is valid but wrong.

1	**DB_HOST**	name the backend uses
2	**DNS**	CoreDNS resolves it
3	**Service**	db-service
4	**Selector**	which pods it picks
5	**Ready endpoints**	pods that pass readiness
6	**targetPort**	where traffic is sent
7	**VXLAN tunnel**	route, firewall, MTU, loss, delay

What we built: diagnose, fix, verify

1

Diagnose

walk the connection path; label the cause with evidence

2

Fix, only if safe

high confidence and one exact action

3

Verify

the app must answer HTTP 200 again

4

Undo or escalate

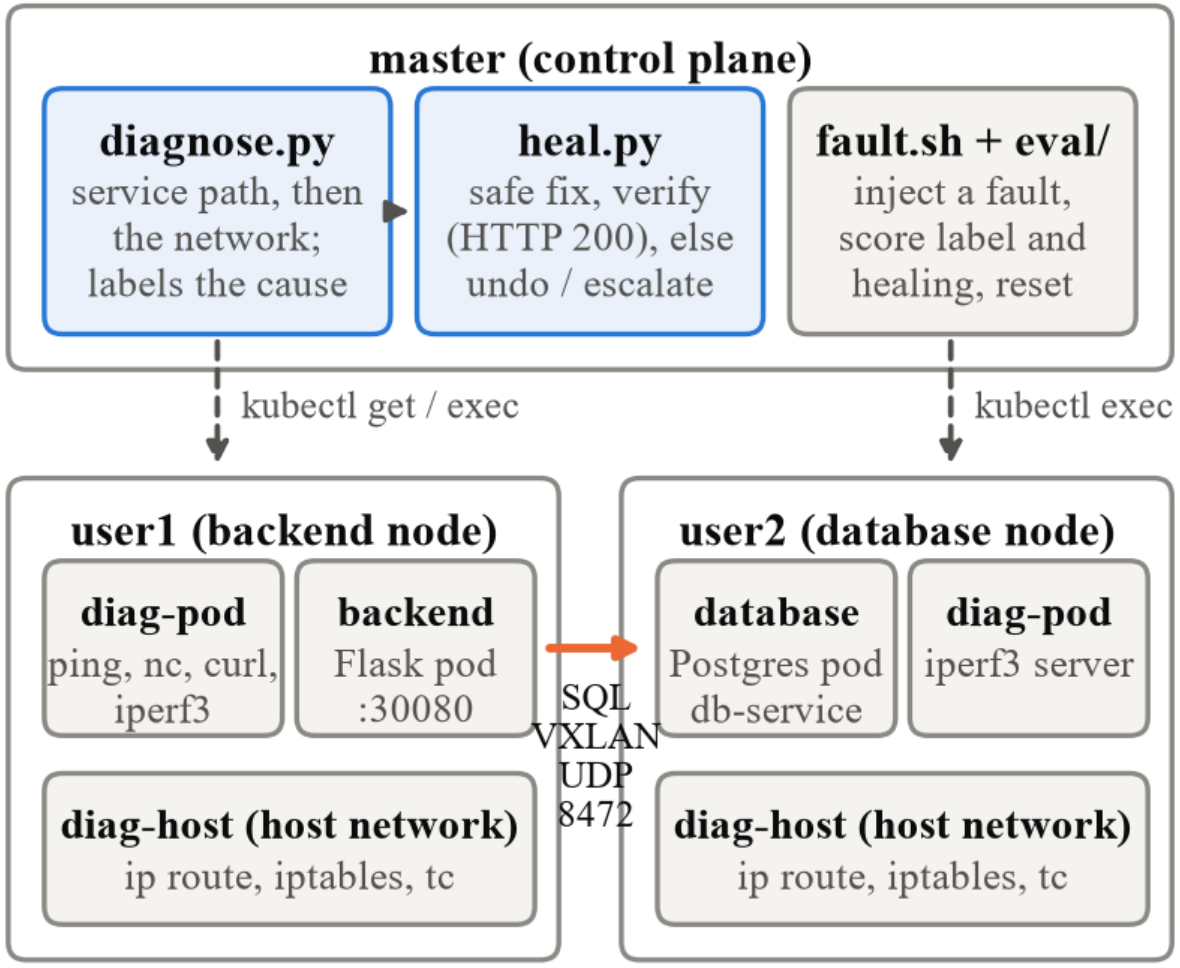
a fix that did not help is rolled back

?

Anything else is labelled **other** and handed to a human, never “fixed”.

A label must hold both ways: the agent reports a fault if and only if it is the cause.

Testbed: three Raspberry Pi 5 on a hotspot



all three Pis on one phone Wi-Fi hotspot (wlan0, 10.206.185.0/24)

Hardware	3× Raspberry Pi 5, 8 GB RAM
Software	Debian 13 (arm64), k3s v1.36, Flannel VXLAN
Network	phone Wi-Fi hotspot (wlan0) only
App	Flask backend on user1 → PostgreSQL 15 on user2
Toolbox	diag-pod (pod network) and diag-host (node network) on every Pi

First, record what healthy looks like

12.7 ms

pod → pod RTT across nodes

(node → node: 10.2 ms)

0 %

packet loss, pod → pod and node → node

1450 B

path MTU in the tunnel

(Wi-Fi: 1500 B, VXLAN header 50 B)

13.2 Mbit/s

pod → pod throughput

(node → node: 14.9 Mbit/s)

All 60 requests through the app succeeded during the baseline (p50 95 ms, p99 162 ms). Wi-Fi signal: −45 to −49 dBm.

Ten faults in two groups; everything else is “other”

Group 1: Kubernetes service path

1. CoreDNS down
2. wrong Service name (DB_HOST)
3. selector does not match the DB pods
4. no Ready endpoints (scaled to 0, failing readiness)
5. wrong targetPort

Group 2: cross-node network

6. VXLAN blocked by a firewall rule (UDP 8472)
7. route to the other node's pods missing
8. MTU black hole (big packets vanish)
9. packet loss on the tunnel
10. high latency on the tunnel

other: a NetworkPolicy blocking the DB, the Wi-Fi itself down, a database error, or a diagnosis that cannot finish (API unreachable)

Diagnosis: follow the path, then compare layers

Group 1: first broken link wins

1. Does CoreDNS have a ready pod?
2. Does the Service in DB_HOST exist?
3. Does its selector pick the DB pods?
4. Is any of them Ready?
5. Is the targetPort the one that is open?

Group 2: compare layers and the baseline

nodes OK, pods ≥ 90 % loss

tunnel broken → check route, firewall, netem

pod loss or RTT ≫ baseline, nodes clean

shaping on the tunnel → netem rule?

1450-byte DF ping fails, small ones pass

MTU black hole → measure path MTU, bulk TCP

tunnel fine, DB pod silent

blocked at the pod → other

Lesson: “refused” does not prove a wrong port

First evaluation run

21/24

All 3 NetworkPolicy trials labelled port_mismatch: k3s rejects denied packets, and the client sees the same “connection refused” as a wrong port.

After requiring positive evidence

24/24

port_mismatch now needs the targetPort to be undeclared or another declared port to be open. Otherwise: other, with the NetworkPolicies listed as evidence.

Healing safely: six rules

1

High confidence, exact fix

otherwise escalate with the evidence

2

One change at a time

each one verified before the next

3

Healed = HTTP 200

not “kubectl succeeded”

4

Undo what did not help

except removing a faulty rule or route

5

Never touch other

a NetworkPolicy may be intentional

6

Log every step

audit trail in heal_log.jsonl

Results on the Pi cluster

24/24

correct labels, group 1 (incl. no-fault and NetworkPolicy → other)

18/18

correct labels, group 2 (incl. no-fault)

30/33

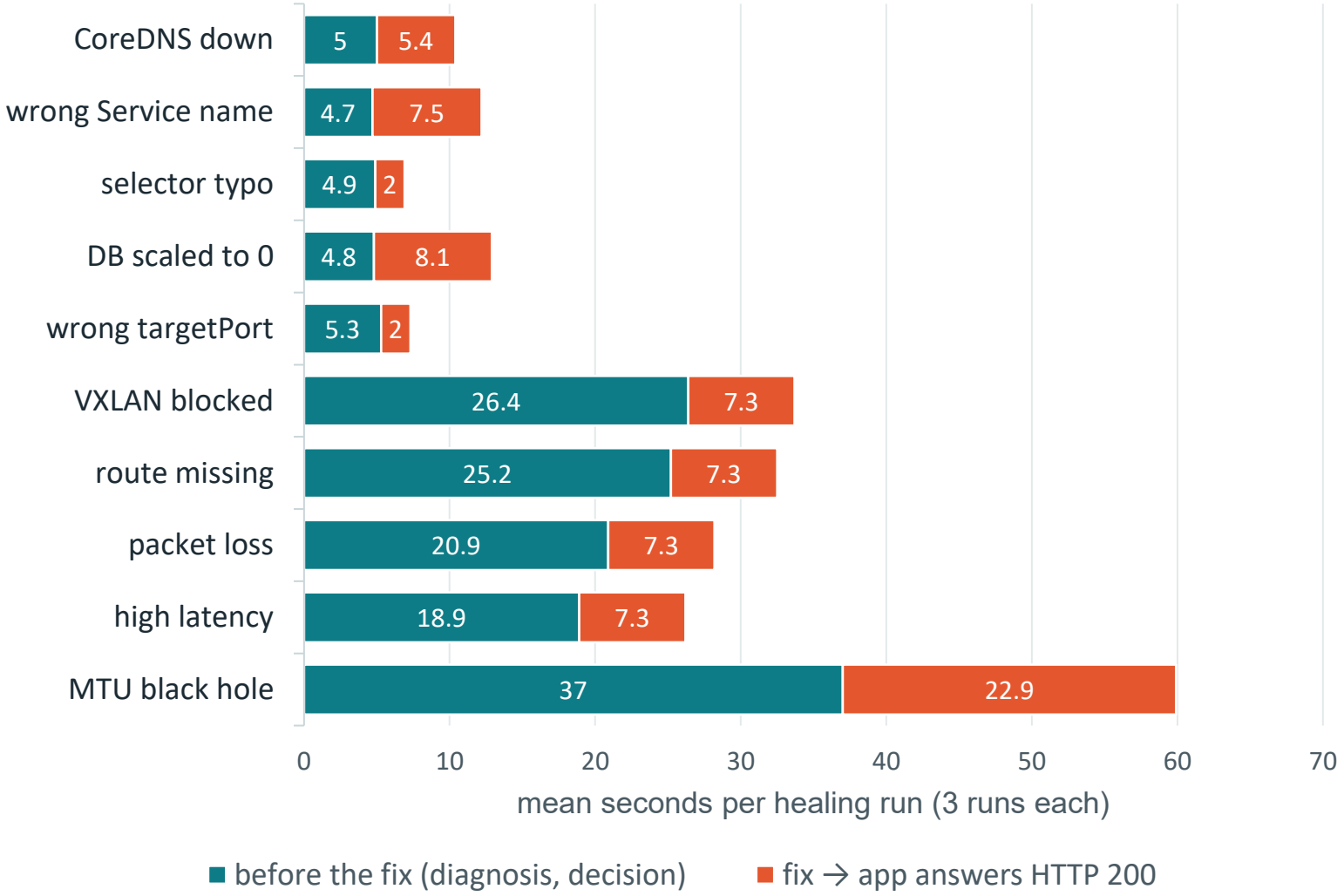
injected faults healed; app answers HTTP 200 again

0

changes in 9 no-fault and uncovered-fault runs

The 3 healing misses: the readiness fault, escalated instead of rolled back by the first version. The rule was fixed (roll back only a revision that was never Ready) but not yet re-run on the Pis.

Time to recover per fault



2–8 s

Kubernetes fixes: from the fix to a working app

≤ 7.3 s

network fixes: one verifying diagnosis

23 s

MTU: verification repeats the MTU search and a bulk transfer

Limitations and next steps

Limitations

- ! one application topology, one fault at a time
- ! faults injected by us: clean signatures
- ! one small cluster on one Wi-Fi network
- ! latest code changes not yet re-run on the Pis

Next steps

- → re-run the full evaluation on the current code
- → explain NetworkPolicy denials (which rule) without changing them
- → handle several simultaneous faults
- → larger clusters and other CNI plugins

Thank you

Questions?

github.com/hemtilva/KuberSelfHeal

code · report · slides · raw results · AI transcripts

Hem Tilva · Vansh Barfiwala · Pratham Choksi · Shivangi Thaker · Jils Shah